

PROGRAMMING II

ABSTRACT CLASSES AND INTERFACES

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026

ABSTRACT CLASSES



ABSTRACT METHOD

An **abstract method** is a class method not having code

- ▶ It describes a functionality provided by the class
- ▶ But it does not provide an implementation for such a functionality

Abstract methods are defined by using the **abstract** keyword

*modifiers **abstract** type methodName(parameters);*

For instance: **public abstract int** computeOnAandB(**int** paramA, **int** paramB);

ABSTRACT CLASS

An **abstract class** is a class containing at least one abstract method

- ▶ Informally, it is an incomplete class
- ▶ Thus, it cannot be instantiated

```
public abstract class Shape { // if there is an abstract method the    1
    private int color;          // class must be abstract                2
    public void setColor(int color) { this.color = color; }            3
                                                                           4
    // the behavior of draw is not specified                             5
    public abstract void draw();                                         6
}                                                                           7
```

Object obj = new Shape(); will raise a compiler error

ABSTRACT CLASS (CONT'D)

An abstract class requires a subclass implementing its abstract methods

- ▶ It can be a concrete class or another abstract class
- ▶ If non-abstract, it can be instantiated *// all methods implemented*

```
public class Circle extends Shape {           1
    // the behavior of draw is specified for this subclass only  2
    public void draw() {                          3
        // code to draw a circle                    4
    }                                              5
}                                              6
}                                              7
```

Object obj = new Circle(); will raise no errors

ABSTRACT CLASS RATIONALE

A superclass can be used to define common behavior for multiple children classes

- ▶ But the class could be too generic to be instantiated
- ▶ Its behavior must be partially unspecified (child-specific)

 An abstract class can extend a concrete class

```
1 public abstract class ArraySorter {
2     // the sorting algorithm should be the same for all array types
3     public final void sort(Object[] arr) {
4         for (int i = 1; i < arr.length; i++)
5             for(int j = 1; j < arr.length; j++)
6                 if (greaterEqual(arr[j-1], arr[j])) {
7                     Object obj = arr[j];
8                     arr[j] = arr[j-1];
9                     arr[j-1] = obj;
10                }
11    }
12    // but the comparison check depends on the type of array elements
13    protected abstract boolean greaterEqual(Object a, Object b);
14 }
```

A SORTER FOR MANY TYPES (CONT'D)

```
public class IntArraySorter extends ArraySorter {           1
    // this implementation is specific for int arrays        2
    @Override                                                3
    protected boolean greaterEqual(Object a, Object b) {    4
        Integer ia = (Integer) a; // here we should have checked the cast! 5
        Integer ib = (Integer) b; // here we should have checked the cast! 6
        return ia >= ib;                                     7
    }                                                         8
}                                                            9
```

IntArraySorter works for Integer[] but not for String[]

- We need to implement a concrete class for String values too

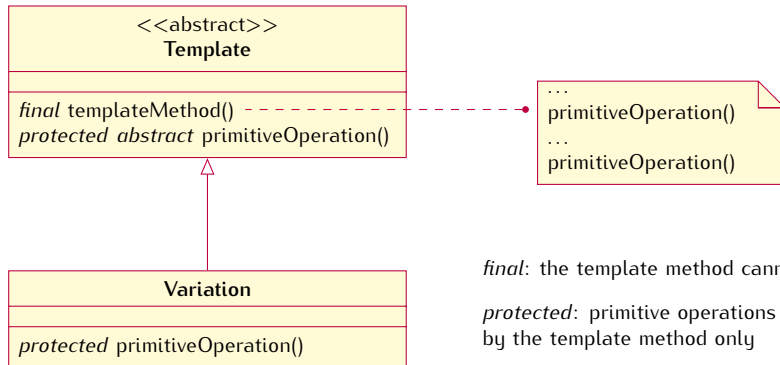
TEMPLATE METHOD PATTERN

Context: A behavior has a stable core and several variations

Problem: Several similar pieces of code must be implemented and maintained

Solution: Separate the core from the variations (template method)

TEMPLATE METHOD PATTERN (CONT'D)



final: the template method cannot be redefined

protected: primitive operations are accessible by the template method only

INTERFACES



INTERFACE

An **interface** is a class containing only abstract methods

- ▶ Thus, it cannot be instantiated
- ▶ Models the behavior of a component, decoupling its implementation

Is an interface a type?

- ▶ Not for objects (no object can be instantiated as an interface)
- ▶ But it is a type for references

🔗 The same applies for abstract classes

INTERFACE (CONT'D)

In an interface all methods and fields are implicitly **public**

- ▶ Methods are also implicitly **abstract**
- ▶ Fields are also implicitly **static** and **final** (constant)

```
interface MyInterface {  
    int A_FIELD = 0;           // public static final  
    void aMethod(String aParam); // public abstract  
    ...  
}
```

1
2
3
4
5

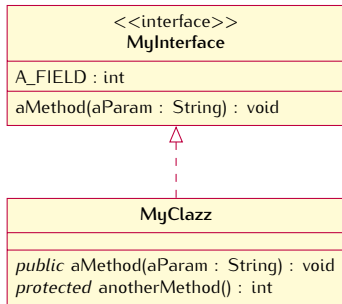
IMPLEMENTING AN INTERFACE

When a class **implements** an interface

- ▶ Must provide an implementation for all methods of the interface
- ▶ Can define additional methods

```
class MyClazz implements MyInterface {           1
    public void aMethod(String aParam) { // here public must be given 2
        if (A_FIELD != Integer.parseInt(aParam)) 3
            System.out.println(aParam);           4
    }                                               5
    protected int anotherMethod() { return 42; }   6
}                                                  7
```

SUBSTITUTION PRINCIPLE AGAIN



MyClazz is a subclass of MyInterface

► `MyInterface ref = new MyClazz();`

```
ref.aMethod("str");           // ok           1
ref.anotherMethod();          // compiler error 2
((MyClazz) ref).anotherMethod(); // ok           3
```

INTERFACES AND INHERITANCE

An interface can extend another interface, but cannot extend a class

```
interface MyInterface extends Comparable { ... }
```

Actually, an interface can extend multiple interfaces

```
interface MyInterface extends Comparable, Orderable { ... }
```

A class can extend only one class, but can implement multiple interfaces

```
class Student extends Person implements Orderable, Comparable { ... }
```


INTERFACE RATIONALE

Interfaces provide complete decoupling between the messages an object can receive and their implementation

- ▶ No shared code, differently from abstract classes
- ▶ Alternative implementations

```
public interface ComplexNumber { 1
    double read();                2
    double imaginary();           3
    double modulus();            4
    double argument();           5
} // can be implemented using either Cartesian or polar coordinates 6
```

COMPARABLE INTERFACE

The `java.lang.Comparable` interface contains the method

// simplified

```
int compareTo(Object obj);
```

Having the following semantics:

- ▶ Returns a **negative** integer if **this** is less than `obj`
- ▶ Returns **zero** if **this** is equal to `obj`
- ▶ Returns a **positive** integer if **this** is greater than `obj`

Used in the `java.util.Arrays` class to sort arrays

COMPARABLE INTERFACE (CONT'D)

```
public class Student extends Person implements Comparable {      1
    ...                                                            2
    @Override                                                       3
    public int compareTo(Object obj) {                               4
        Student other = (Student) obj; // we should check the cast 5
        return this.matricola - other.matricola;                   6
    }                                                                7
}                                                                    8
                                                                    9

Student[] arrS = {new Student("Sam", 456), new Student("Paul", 123)}; 10
Person[] arrP = {new Person("Sam"), new Person("Paul")};           11
                                                                    12

Arrays.sort(arrS); // ok -- array sorted                            13
Arrays.sort(arrP); // runtime error -- ClassCastException           14
                    // Person cannot be casted to java.lang.Comparable 15
```

ITERABLE AND ITERATOR INTERFACES

The `java.lang.Iterable` interface contains the method

// simplified

```
Iterator iterator();
```

A class implementing `Iterable` can be used in enhanced for loops

- ▶ The interface uses the `Iterator` interface
- ▶ To iterate on a collection of elements

ITERABLE AND ITERATOR INTERFACES (CONT'D)

The `java.util.Iterator` interface contains the methods

// simplified

```
boolean hasNext(); and Object next();
```

Having the following semantics:

- ▶ Start iteration before the first element
- ▶ `hasNext()` tells if a next element is present
- ▶ `next()` returns the next element and advances by one position

ITERABLE AND ITERATOR INTERFACES (CONT'D)

```
import java.util.Iterator;
import java.lang.Iterable;

class Classroom implements Iterable {
    private Student[] students;

    Classroom(Student[] students) {
        this.students = students;
    }
    @Override
    public Iterator iterator() {
        return new CRIterator(students);
    }
} // we could have implemented this
// class more elegantly by using
// an inner class
```

```
import java.util.Iterator;

class CRIterator implements Iterator {
    private Student[] arr;
    private int next = 0;

    CRIterator(Student[] arr) { this.arr = arr; }
    @Override
    public boolean hasNext() {
        return next < arr.length;
    }
    @Override
    public Object next() {
        return new Student(arr[next++]);
    }
}
```

```
Student[] arrS = {new Student("Sam", 456), new Student("Paul", 123)};
Classroom classRoom = new Classroom(arrS);
for (Object obj : classRoom) // to make the code more elegant we need generics
    System.out.println(((Student) obj).toString)
```



Q + A

